

Lab Session 12

Introduction to GitHub: Repository Management and Version Control

1. Introduction

In modern software development, **version control** is a critical skill. Version control systems allow developers to **track changes, collaborate, and maintain the history of code**. **Git** is the most widely used version control system, and **GitHub** is a cloud-based platform that hosts Git repositories, providing collaboration tools, issue tracking, and online storage.

This lab introduces students to **GitHub fundamentals** and basic Git commands. Students will create repositories, manage files, commit changes, push to GitHub, and understand the workflow of version control.

2. Objectives

After completing this lab, students will be able to:

1. Understand the role of Git and GitHub in software development.
2. Create a new GitHub repository.
3. Clone a repository to a local machine.
4. Stage, commit, and push changes to GitHub.
5. Pull updates from a remote repository.
6. Understand branching and basic collaboration concepts.

3. Theory

3.1 Git and GitHub

- **Git** is a distributed version control system that tracks changes in files.
- **GitHub** is a web-based platform that hosts Git repositories, enabling collaboration.
- **Local repository**: The copy of a repository on your computer.
- **Remote repository**: The repository hosted online (GitHub).

3.2 Basic Git Workflow

1. **Clone a repository** → get a local copy.
2. **Edit files locally** → make changes.
3. **Stage changes** → mark changes to be committed.
4. **Commit changes** → save changes to local Git history.
5. **Push changes** → update the remote repository.
6. **Pull changes** → fetch updates from remote to local.

4. Tools and Resources

- **Git** (install if not already: <https://git-scm.com/>)
- **GitHub account** (<https://github.com/>)
- **Text editor** (VS Code recommended)
- **Terminal/Command Prompt**

5. Lab Procedure

Step 1: Set Up Git on Your Machine

1. Check if Git is installed:

```
</> Bash  
git --version
```

- Output example: git version 2.42.0
- If Git is not installed, download and install from <https://git-scm.com/downloads>

2. Configure Git with your username and email:

```
</> Bash  
git config --global user.name "Your Name"  
git config --global user.email "youremail@example.com"
```

Explanation: Git uses this information to track commits.

Step 2: Create a GitHub Repository

1. Log in to GitHub.
2. Click “**New Repository**”.
3. Enter repository name, e.g., MyFirstRepo.
4. Select **Public** or **Private**.
5. Initialize with a **README** (optional).
6. Click “**Create Repository**”.

Explanation:

- A repository is a storage space for your project.
- Public repositories can be seen by anyone; private are restricted.

Step 3: Clone the Repository Locally

1. Copy the repository URL from GitHub (HTTPS recommended).
2. Open terminal/command prompt.
3. Run:

```
</> Bash  
  
git clone https://github.com/username/MyFirstRepo.git
```

Explanation:

- Cloning creates a local copy of the repository.
- Students can now edit files locally.

Step 4: Add a File and Stage Changes

1. Navigate into the repository folder:

```
</> Bash  
  
cd MyFirstRepo
```

2. Create a file, e.g., index.html.
3. Add some content:

```
</> HTML  
  
<!DOCTYPE html>  
<html>  
<head>  
  <title>My First Repo</title>  
</head>  
<body>  
  <h1>Hello, GitHub!</h1>  
</body>  
</html>
```

4. Stage the file:

```
</> Bash  
  
git add index.html
```

Explanation:

- `git add` tells Git to include this file in the next commit.

Step 5: Commit Changes

```
</> Bash  
git commit -m "Add index.html with basic HTML structure"
```

Explanation:

- `git commit` records changes in the local repository.
- `-m` allows adding a commit message describing the changes.

Step 6: Push Changes to GitHub

```
</> Bash  
git push origin main
```

Explanation:

- `git push` uploads local commits to the remote repository.
- `origin` refers to the remote repository.
- `main` is the default branch name (used to be master).

Step 7: Pull Updates from Remote Repository

```
</> Bash  
git pull origin main
```

Explanation:

- `git pull` fetches updates from the remote repository and merges them into the local repository.
- Important when multiple people work on the same project.

Step 8: Branching (Optional Advanced Step)

1. Create a new branch:

```
</> Bash  
git checkout -b feature-branch
```

2. Make changes, stage, and commit.
3. Push the branch:

```
</> Bash  
git push origin feature-branch
```

4. On GitHub, create a pull request to merge the branch into main.

Explanation:

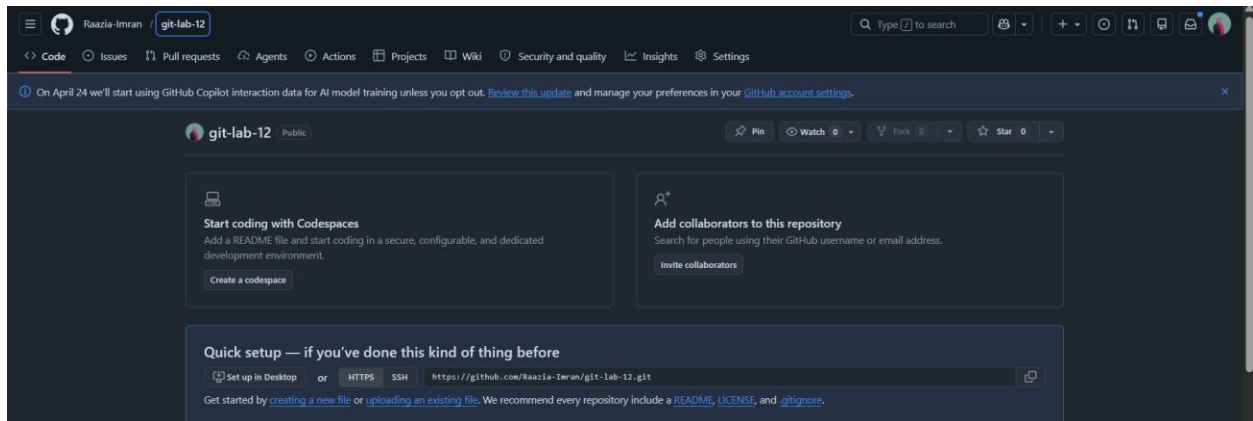
- Branches allow developers to work on new features without affecting the main code.
- Pull requests are used to review and merge changes.

Task 1

Create a GitHub repository and clone it to your machine.

```
C:\Users\hp>git --version
git version 2.47.1.windows.2

C:\Users\hp>
```



```
C:\Users\hp>git clone https://github.com/Raazia-Imran/git-lab-12
Cloning into 'git-lab-12'...
warning: You appear to have cloned an empty repository.

C:\Users\hp>|
```

```
C:\Users\hp>cd git-lab-12

C:\Users\hp\git-lab-12>dir
Volume in drive C has no label.
Volume Serial Number is 10DF-278D

Directory of C:\Users\hp\git-lab-12

22/04/2026  05:36 pm    <DIR>          .
22/04/2026  05:36 pm    <DIR>          ..
                0 File(s)                0 bytes
                2 Dir(s)  40,210,300,928 bytes free

C:\Users\hp\git-lab-12>|
```

Task 2

Add a new file, stage it, commit it, and push it to GitHub.

```
C:\Users\hp\git-lab-12>notepad lab1.txt

C:\Users\hp\git-lab-12>dir
Volume in drive C has no label.
Volume Serial Number is 10DF-278D

Directory of C:\Users\hp\git-lab-12

22/04/2026  05:41 pm    <DIR>          .
22/04/2026  05:36 pm    <DIR>          ..
22/04/2026  05:41 pm                30 lab1.txt
                1 File(s)                30 bytes
                2 Dir(s)  40,209,166,336 bytes free

C:\Users\hp\git-lab-12>|
```

```
C:\Users\hp\git-lab-12>git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    lab1.txt

nothing added to commit but untracked files present (use "git add" to track)
C:\Users\hp\git-lab-12>|
```

```
C:\Users\hp\git-lab-12>git add lab1.txt

C:\Users\hp\git-lab-12>git commit -m "Add lab1.txt file"
[main (root-commit) dc9cf48] Add lab1.txt file
 1 file changed, 1 insertion(+)
 create mode 100644 lab1.txt

C:\Users\hp\git-lab-12>git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 248 bytes | 82.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Raazia-Imran/git-lab-12
 * [new branch]      main -> main

C:\Users\hp\git-lab-12>|
```

Task 3

Modify the file and push the update. Observe changes on GitHub.

```
C:\Users\hp\git-lab-12>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   lab1.txt

no changes added to commit (use "git add" and/or "git commit -a")
C:\Users\hp\git-lab-12>|
```

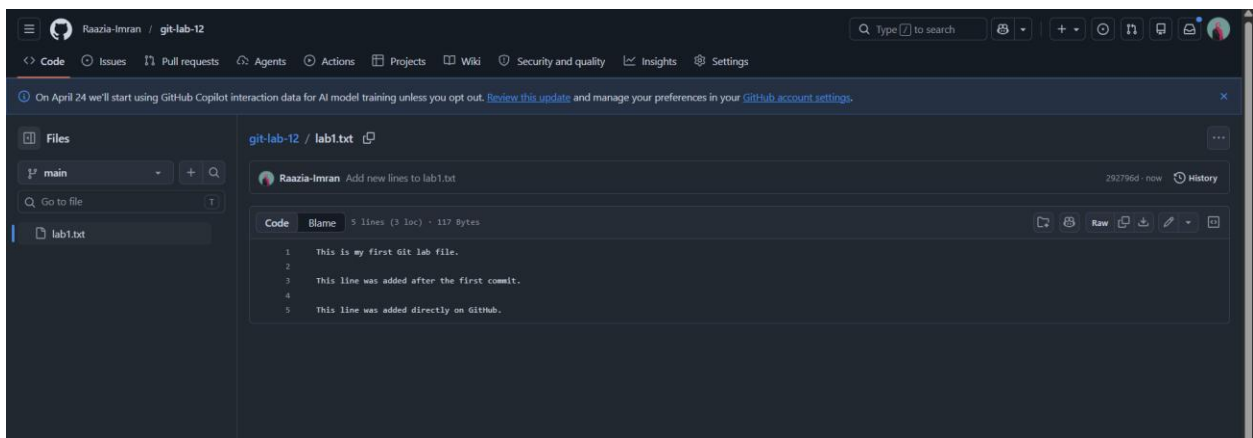
```

C:\Users\hp\git-lab-12>git commit -m "Update lab1.txt with new line"
[main 1918764] Update lab1.txt with new line
 1 file changed, 3 insertions(+), 1 deletion(-)

C:\Users\hp\git-lab-12>git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 328 bytes | 328.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Raazia-Imran/git-lab-12
dc9cf48..1918764  main -> main

C:\Users\hp\git-lab-12>

```



Task 4

Pull updates from remote if changes are made online.

```

C:\Users\hp\git-lab-12>git pull origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 983 bytes | 61.00 KiB/s, done.
From https://github.com/Raazia-Imran/git-lab-12
 * branch                main          -> FETCH_HEAD
 1918764..292796d  main          -> origin/main
Updating 1918764..292796d
Fast-forward
 lab1.txt | 4 +++-
 1 file changed, 3 insertions(+), 1 deletion(-)

C:\Users\hp\git-lab-12>

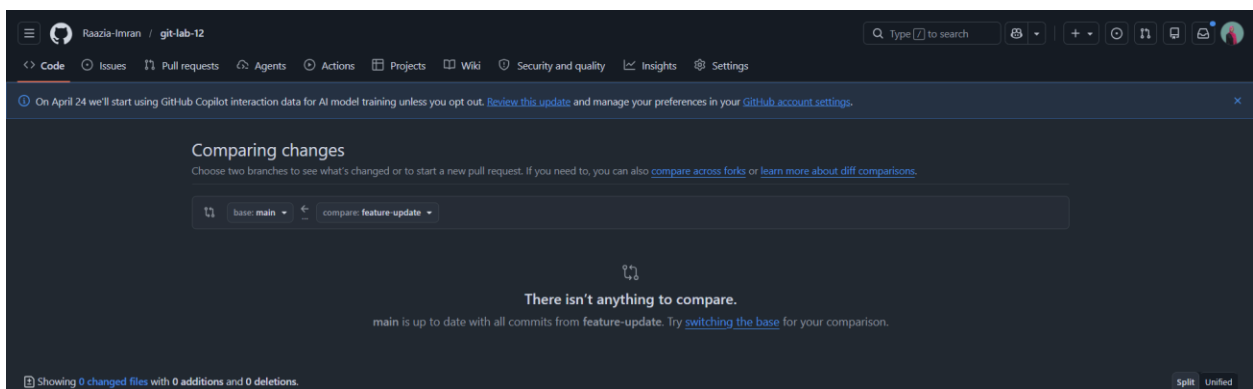
```

Task 5

Optional: Create a new branch, make changes, push it, and open a pull request.

```
C:\Users\hp\git-lab-12>git push origin feature-update
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 1.42 KiB | 485.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
remote:
remote: Create a pull request for 'feature-update' on GitHub by visiting:
remote:      https://github.com/Raazia-Imran/git-lab-12/pull/new/feature-update
remote:
To https://github.com/Raazia-Imran/git-lab-12
 * [new branch]      feature-update -> feature-update

C:\Users\hp\git-lab-12>|
```



Task 6

Explain the purpose of each Git command used in the lab.

Command	Purpose
git clone	Copies a remote repository to the local machine
git status	Shows the current state of files
git add	Stages files for commit
git commit	Saves staged changes with a message
git push	Uploads commits to GitHub
git pull	Fetches and merges changes from remote
git checkout -b	Creates and switches to a new branch